OS 5th Semester 21CS Project

A Mini Multi-Threaded 21CS Chat Room Created in Java

Muhammad Sameer	M. Mashaal Zulfikar
21CS077	21CS105

Introduction

In today's digital age, real-time communication has become an integral part of our daily lives. Whether it's for socializing, collaboration, or business purposes, the need for creating interactive chat applications has become both feasible and popular.

The **Mini Multi-Threaded Chat Room** project aims to demonstrate the implementation of a simple yet functional chat room system using Java programming language. This project serves as an educational tool for understanding concepts such as socket programming, multi-threading, and client-server architecture.

Objectives

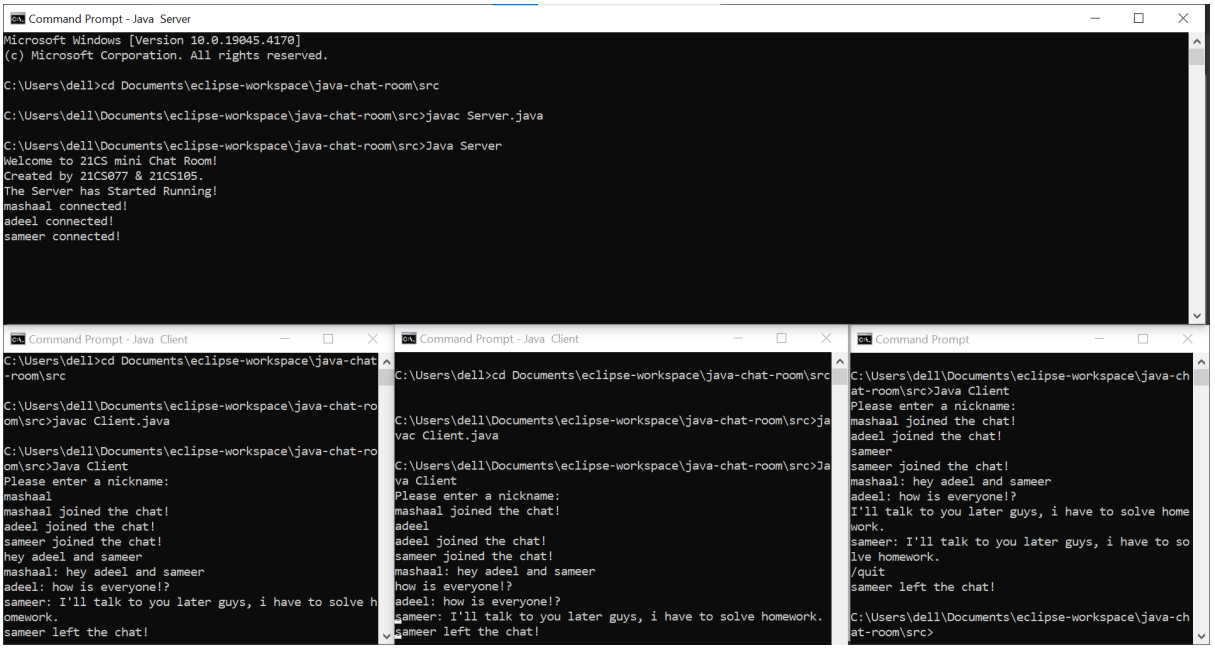
The primary objectives of this project are as follows:

1. **Implementation**: Develop a server-client architecture for real-time messaging using Java sockets (Java sockets provide a mechanism for communication between two endpoints over a network, enabling the exchange of data between client and server applications.).
2. **Concurrency**: Utilize multi-threading to handle multiple client connections concurrently without blocking.
3. **Functionality**: Implement essential chat room functionalities such as sending and receiving messages, changing nicknames, and gracefully handling client disconnects.

Background

We came up with the idea for this project because we wanted to learn more about how computers communicate over networks and how multiple things can happen at the same time in a computer program. Specifically, we wanted to explore these concepts using Java, a programming language we're familiar with.

By making a chat room where lots of people can talk to each other at the same time, we aimed to understand how to handle many people connecting to a computer server all at once, how to make sure messages get to the right people, and how to make sure the chat room works well even when lots of people are using it.



Three clients connected to the same server chatting with each other.

Real-Life Examples of Multi-Threaded Chat Rooms

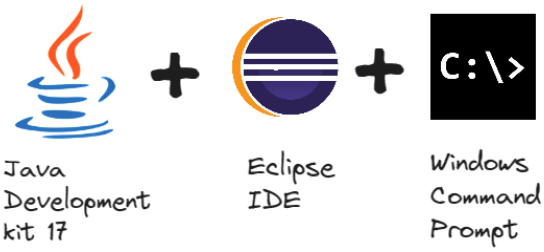
Real-life examples of projects similar to a multi-threaded chat room can be found in various messaging applications and online communication platforms:

1. **Messaging Apps**: Applications like WhatsApp, Discord, and Telegram are prime examples. These apps allow users to send messages to each other in real-time, just like a chat room. They handle multiple users connecting simultaneously and manage message exchanges efficiently.
2. **Social Media Platforms**: Platforms like Instagram and Snapchat, and private organization offices where employees or clients connect to a single server

of the office to send some information or notifications to each other for a process updates, also involve real-time messaging features. Users can comment on posts, reply to each other's comments, and send direct messages, all of which require efficient handling of concurrent connections and message exchanges.

These various messaging applications and online communication platforms build servers and provide an application to the clients to facilitate real-time messaging, enabling users to connect, exchange messages, and interact seamlessly across different devices and platforms.

Software and Applications Used

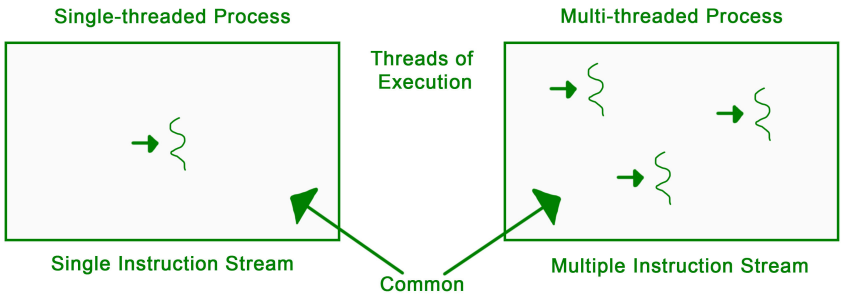


- **Java JDK 17:** A software development kit used to develop Java applications. Version 17 is one of the recent versions of JDK, offering tools for compiling, debugging, and running Java programs, as well as libraries and APIs for building various types of applications.
- **Eclipse IDE:** A popular integrated development environment (IDE) used for Java development. It offers a wide range of features, including code editing, debugging, version control integration, and project management.
- **Command Prompt:** Also known as Command Line Interface (CLI). A text-based interface provided by operating systems like Windows. It allows users to interact with the operating system and execute commands by typing text commands.

  In the context of Java development, the command prompt is often used to compile Java source code files (.java) into bytecode files (.class) using the `javac` command and to run Java applications using the `java` command.

Concept of Multithreading in OS

In an operating system, multithreading refers to the ability of a CPU to execute multiple threads concurrently within a single process. A thread is a lightweight process that shares the same memory space and resources as other threads within the same process. Multithreading allows for better utilization of CPU resources and improved responsiveness in handling multiple tasks simultaneously.



Single Thread and Multi Thread Process

Threading is used widely in almost every field. Most widely it is seen over the internet nowadays where we are using real-time communication or transaction processing of every type, like messaging application, online data transfer, cautious transaction update processes etc. Threading is a segment which divide the code into small parts that are of very light weight and has less burden on CPU memory so that it can be easily worked out and can achieve goal in desired field with less execution time possible.

Execution in the Java Program:

In our Java program, we've implemented a multi-threaded chat room system. Here's how the concept of multithreading is relevant in the context of our program:

1. **Server-side Execution**: https://www.dropbox.com/scl/fi/33n8qtsfacov8qrqlf97e/Server-Java.txt?rlkey=g9mwo5jml66kuip19j60kwuwg&dl=0

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
```

```
import java.net.Socket;
import java.util.ArrayList;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Server implements Runnable {

    private final ArrayList<ConnectionHandler> connections
    private ServerSocket server;
    private boolean done;
    private ExecutorService pool;

    public Server() {
        done = false;
        connections = new ArrayList<>();
    }

    @Override
    public void run() {
        try {
            server = new ServerSocket(9999);
            pool = Executors.newCachedThreadPool();
            while (!done) {
                Socket client = server.accept();
                ConnectionHandler handler = new Connection
                connections.add(handler);
                pool.execute(handler);
            }
        } catch (Exception e) {
            shutdown();
        }
    }

    public void broadcast(String message) {
        for (ConnectionHandler ch : connections) {
            if (ch != null) {
                ch.sendMessage(message);
            }
```

```
        }
    }

    public void shutdown() {
        try {
            done = true;
            pool.shutdown();
            if (!server.isClosed()) {
                server.close();
            }
            for (ConnectionHandler ch : connections) {
                ch.shutdown();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

    }

    class ConnectionHandler implements Runnable {

        private final Socket client;
        private BufferedReader in;
        private PrintWriter out;

        public ConnectionHandler(Socket client) {
            this.client = client;
        }

        @Override
        public void run() {
            try {
                out = new PrintWriter(client.getOutputStre
                in = new BufferedReader(new InputStreamRea

                // Prompt the client for a nickname until 
                String nickname;
                do {
```

```
                    out.println("Please enter a nickname:
                    nickname = in.readLine();
                } while (nickname == null || nickname.trim

                System.out.println(nickname + " connected!
                broadcast(nickname + " joined the chat!");
                String message;
                while ((message = in.readLine()) != null)
                    if (message.startsWith("/nick")) {
                        String[] messageSplit = message.sp
                        if (messageSplit.length == 2) {
                            broadcast(nickname + " renamed
                            System.out.println(nickname +
                            nickname = messageSplit[1];
                            out.println("Successfully chan
                        } else {
                            out.println("No nickname provi
                        }
                    } else if (message.startsWith("/quit")
                        broadcast(nickname + " left the ch
                        shutdown();
                    } else {
                        broadcast(nickname + ": " + messag
                    }
                }
            } catch (IOException e) {
                shutdown();
            }
        }

        public void sendMessage(String message) {
            out.println(message);
        }

        public void shutdown() {
            try {
                in.close();
                out.close();
```

```
                if (!client.isClosed()) {
                    client.close();
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }

    }

    public static void main(String[] args) {
        System.out.println("Welcome to 21CS mini Chat Room
        System.out.println("This Program is created by 21C
        System.out.println("The Server has Started Running
        Server server = new Server();
        server.run();
    }
}
```

- The main server class (`Server`) listens for incoming client connections using a `ServerSocket`.
- When a client connects, the server creates a new `ConnectionHandler` thread to handle communication with that client while continuing to listen for other connections.
- This allows the server to handle multiple client connections concurrently without blocking, enabling seamless communication between clients without waiting for one client's operation to finish before serving another.

2. **Client-side Execution**: https://www.dropbox.com/scl/fi/9xb1uq88cyala5sspp93p/Client-Java.txt?rlkey=i8yr2rsos8mw5b2ax67dz7zsx&dl=0

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
```

```
import java.net.Socket;

public class Client implements Runnable{
    private Socket client;
    private BufferedReader in;
    private PrintWriter out;
    private boolean done;
    @Override
    public void run() {
        try{
            client = new Socket("127.0.0.1",9999);
            out = new PrintWriter(client.getOutputStream()
            in = new BufferedReader(new InputStreamReader(

            InputHandler inHandler = new InputHandler();
            Thread t = new Thread(inHandler);
            t.start();

            String inMessage;
            while ((inMessage = in.readLine()) != null){
                System.out.println(inMessage);
            }

        }catch (IOException e){
            shutdown();
        }
    }

    public void shutdown(){
        done = true;
        try {
            in.close();
            out.close();
            if(!client.isClosed()){
                client.close();
            }
        }catch (IOException e){
            e.printStackTrace();
```

```
        }
    }

    class InputHandler implements Runnable{
        @Override
        public void run() {
            try{
                BufferedReader inReader = new BufferedRead
                while (!done){
                    String message = inReader.readLine();
                    if(message.equals("/quit")){
                        out.println(message);
                        inReader.close();
                        shutdown();
                    }else {
                        out.println(message);
                    }
                }
            }catch (IOException e){
               shutdown();
            }
        }
    }

    public static void main(String[] args) {
        Client client = new Client();
        client.run();
    }
}
```

- Each client (represented by the `Client` class) also operates using multiple threads.
- One thread (`InputHandler`) is responsible for reading input from the user and sending it to the server.
- Another thread is dedicated to listening for messages from the server and printing them to the client's console.

- This concurrent execution allows the client to send messages while simultaneously receiving and displaying messages from other clients in real-time.

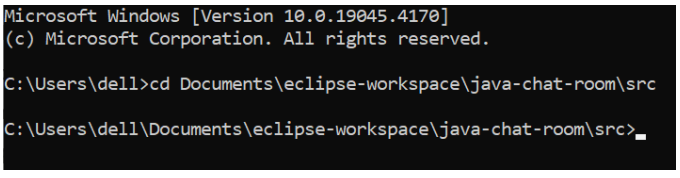
Relevance to Multithreading in Operating Systems:

- **Concurrency**: In both the server and client sides of our Java program, multiple threads are executing concurrently, just like in multithreading within an operating system.
- **Resource Sharing**: Threads within the same process (either server or client) share resources such as sockets, input/output streams, and memory space, similar to how threads within a process share resources in an operating system. To make this chat room, we used the `Runnable` interface to describe the SErver and Client class whose instances can be run as a thread.
- **Efficiency**: By leveraging multithreading, our program can handle multiple client connections efficiently, ensuring responsiveness and scalability, which aligns with the benefits of multithreading in operating systems.

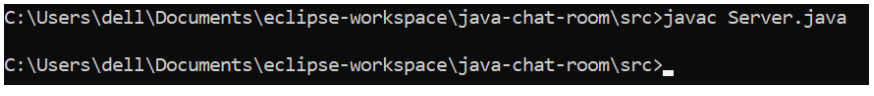
How to Run the Program on Command Prompt:

Initiating the Server

- Open Command Prompt
- Type **cd Documents\eclipse-workspace\java-chat-room\src** access the source code file.



- Now type **javac Server.java** to compile the Server class.



- Now run the server by typing **Java Server**.

```
C:\Users\dell\Documents\eclipse-workspace\java-chat-room\src>Java Server
Welcome to 21CS mini Chat Room!
Created by 21CS077 & 21CS105.
The Server has Started Running!
```

Initiating the Clients

- Open Command Prompt
- Type **cd Documents\eclipse-workspace\java-chat-room\src** access the source code file.

```
Microsoft Windows [Version 10.0.19045.4170]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dell>cd Documents\eclipse-workspace\java-chat-room\src

C:\Users\dell\Documents\eclipse-workspace\java-chat-room\src>
```

- Now type **javac Client.java** to compile the Server class.

```
C:\Users\dell\Documents\eclipse-workspace\java-chat-room\src>javac Client.java

C:\Users\dell\Documents\eclipse-workspace\java-chat-room\src>
```

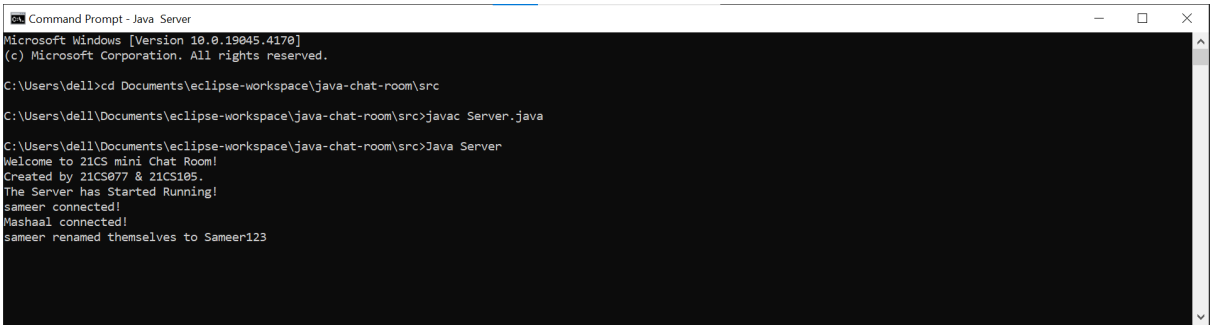
- Now run the server by typing **Java Client**.

```
C:\Users\dell\Documents\eclipse-workspace\java-chat-room\src>Java Client
Please enter a nickname:
```

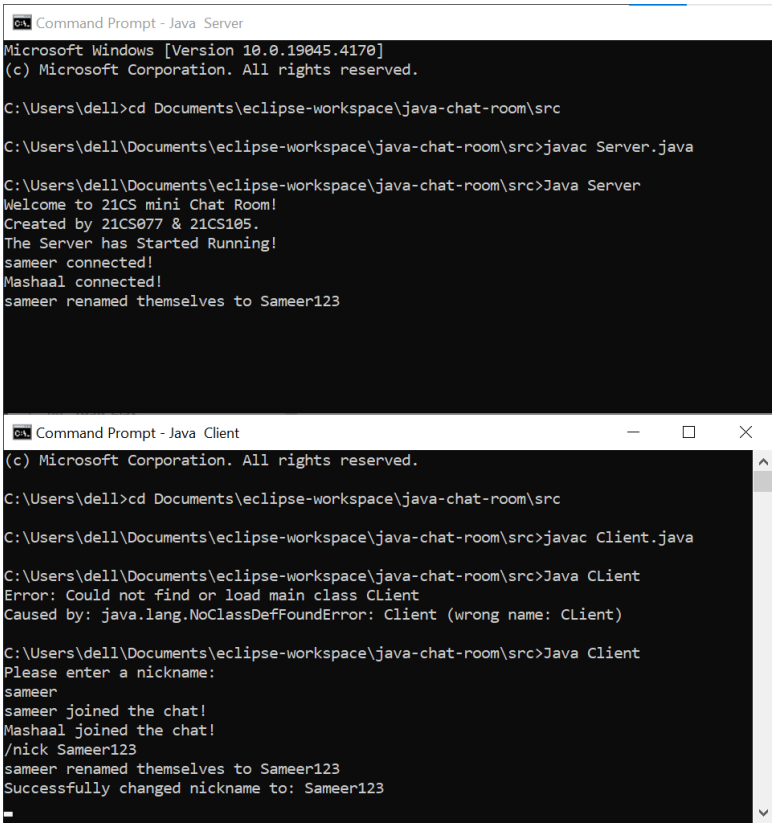
Scope of the Project

This project primarily focuses on the core functionality of establishing communication between multiple clients and a central server. It does not aim to provide advanced features like encryption, user authentication, or graphical user interfaces. The scope includes:

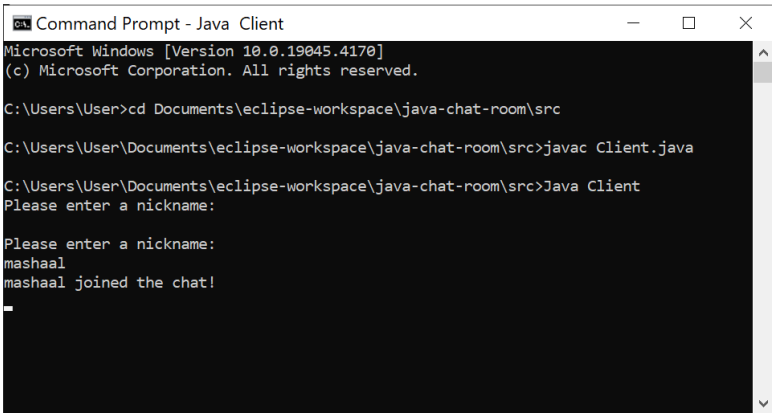
- Establishing TCP connections between clients and the server.



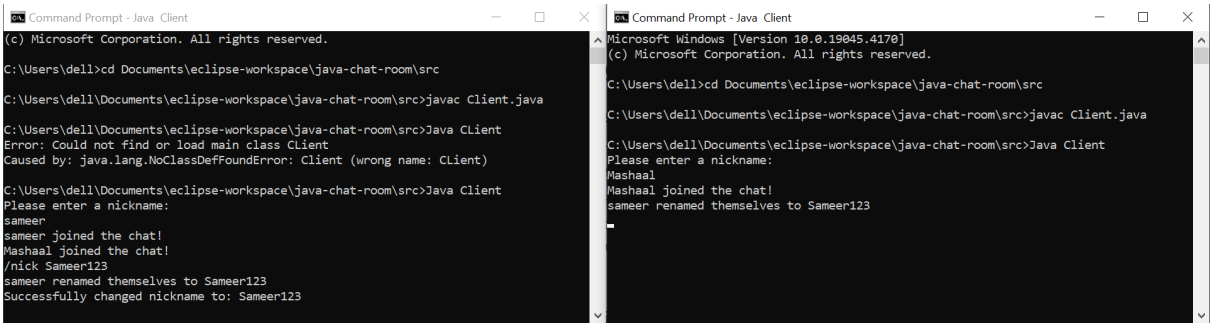
- Handling message exchanges and basic commands (e.g., changing nickname, quitting).



- Handling IOException, for e.g. the Client window forces the client to enter a nickname rather than being in the chat room without a nickname. Sometimes, when the server is lost while the client is trying to send a message, it may result in an `IOException` as well.



- Managing multiple client connections concurrently using multi-threading.



Limitations of the Program

Limitation 1: Lack of Authentication and Security

- **Limitation**: The chat room currently lacks authentication mechanisms, making it vulnerable to unauthorized access and potential security threats.
- **Solution**: Implement user authentication mechanisms such as username/password authentication or token-based authentication to ensure that only authorized users can access the chat room. Additionally, consider implementing encryption for secure communication between clients and the server.

Limitation 2: Lack of Persistence

- **Limitation**: The chat room does not persist messages or user data, leading to data loss if the server is restarted or crashes.
- **Solution**: Integrate a database or file storage system to persist messages and user data. This allows for message history retrieval, user management, and recovery in case of server failures.

Limitation 3: Limited User Interface

- **Limitation**: The chat room currently has a basic text-based interface, which may not be user-friendly or visually appealing.
- **Solution**: Develop a graphical user interface (GUI) using JavaFX or Swing to provide a more intuitive and visually appealing interface for users. This can include features such as message formatting, emoticons, and user avatars.

Conclusion

In conclusion, the multi-threaded chat room project in Java has provided valuable insights into the complexities and nuances of network programming, concurrent execution, and application development. Through the implementation and analysis of the project, several key takeaways emerge:

- **Understanding of Multithreading**: The project deepened our understanding of multithreading concepts, particularly in the context of managing multiple client connections concurrently within a single server process. We learned how to leverage Java's concurrency mechanisms to create a responsive and scalable chat room system.
- **Practical Application of Networking**: By developing a client-server architecture, we gained practical experience in socket programming and network communication protocols. We learned how to establish and manage TCP/IP connections between clients and a central server, enabling real-time message exchanges in the chat room application.
- **Challenges and Solutions**: Throughout the project, we encountered various challenges, such as security concerns, and error handling issues. However, by implementing solutions such as thread pooling, authentication mechanisms, and robust error handling, we were able to mitigate these challenges and enhance the overall functionality and reliability of the system.
- **Future Directions**: While the current implementation provides a solid foundation for a basic chat room application, there are opportunities for further enhancement and refinement. Future iterations of the project could focus on implementing additional features such as message persistence, user authentication, encryption, and a more user-friendly graphical interface.